

Web and Data Engineering

Kapitel 3: Frontend-Technologien — HTML, JavaScript und CSS

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Jede Web-Anwendung, die ein Nutzer sieht und bedient, besteht aus drei Technologien: HTML für Struktur, CSS für Darstellung und JavaScript für Verhalten. Diese Vorlesung vermittelt die **Grundlagen** aller drei — nicht als Programmierkurs, sondern als **Architekturwissen** für Web Engineers.

Leitfragen

- Warum ist semantisches HTML mehr als Markup?
- Wie macht JavaScript den Browser zur Laufzeitumgebung?
- Wie erzeugt CSS aus derselben Struktur unterschiedliche Layouts?
- Wie arbeiten HTML, CSS und JavaScript im Browser zusammen?

HTML — Struktur und Semantik

Leitfrage: Warum reicht es nicht, eine Web-Seite einfach mit `<div>`-Elementen zusammenzubauen — und was gewinnt man durch semantisches HTML?

Zwei Versionen derselben Seite

Version A

```
<div class="top">
  <div class="logo">Shop</div>
  <div class="links">
    <div><a href="/">Start</a></div>
    <div><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="middle">
  <div class="left">
    <div class="title">Funkkopfhörer</div>
    <div>Kabelloser Kopfhörer mit...</div>
    <div class="price">€ 79,99</div>
  </div>
  <div class="right">
    <div>Auch interessant: ...</div>
  </div>
</div>
<div class="bottom">© 2026 Shop</div>
```

Version B

```
<header>
  <h1>Shop</h1>
  <nav>
    <a href="/">Start</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h2>Funkkopfhörer</h2>
    <p>Kabelloser Kopfhörer mit...</p>
    <p class="price">€ 79,99</p>
  </article>
  <aside>Auch interessant: ...</aside>
</main>
<footer>© 2026 Shop</footer>
```

Aufgabe: Beide Versionen sehen im Browser identisch aus. Was unterscheidet sie — und für wen?

Was Version B besser macht

Nutzer	Version A (<div>)	Version B (semantisch)
Screenreader	„div, div, div, Link, div, div..." — keine Orientierung	„Navigation mit 2 Links, Hauptinhalt: Artikel ‚Funkkopfhörer‘, Sidebar"
Suchmaschine	Muss raten, was Hauptinhalt ist	<article> + <h2> = klares Inhaltsranking
Entwickler	Muss CSS-Klassen lesen, um Struktur zu verstehen	Code ist selbstdokumentierend
Browser	Kein Reader-Mode, keine sinnvolle Druckansicht	Reader-Mode extrahiert <article>, Druckansicht setzt <main>

Kernprinzip

HTML trennt

- **Struktur** (was ist ein Absatz, eine Überschrift, eine Navigation?)
- von **Darstellung** (wie sieht es aus? → CSS)
- und **Verhalten** (was passiert bei Klick? → JavaScript).

Das Technologie-Dreieck des Webs

Jede Web-Anwendung besteht aus drei Schichten mit klarer Verantwortungsteilung:

Schicht	Technologie	Verantwortung	Leitfrage
Struktur	HTML	Was existiert auf der Seite?	Welche Elemente, welche Bedeutung?
Darstellung	CSS	Wie sieht es aus und wo steht es?	Farbe, Layout, Animation, Responsivität
Verhalten	JavaScript	Was passiert bei Interaktion?	DOM-Änderungen, Server-Kommunikation, Zustand

Warum drei getrennte Technologien?

Dasselbe HTML kann mit **verschiedenem CSS** völlig anders aussehen — dieselbe Shop-Seite als Desktop-Layout, als Mobilansicht, als Druckversion. JavaScript kann **DOM und CSSOM verändern**, aber HTML und CSS können auch **ohne JavaScript** funktionieren: Links navigieren, Formulare senden, Hover-Effekte reagieren.

Diese Schichtung ist kein Zufall — sie ist das Architekturprinzip des Webs.

Hinter den Standards: W3C, WHATWG und die Standardisierung des Webs

Die drei Frontend-Technologien sind keine Produkte einer einzelnen Firma, sondern Ergebnis von Standardisierung.

- **WHATWG** (*Web Hypertext Application Technology Working Group*) ist eine von Browser-Herstellern geprägte Arbeitsgruppe und pflegt heute den **HTML Living Standard** sowie das **DOM**.
- **W3C** (*World Wide Web Consortium*) ist das breitere Standardisierungskonsortium des Webs; dort liegen u. a. **CSS** und **ARIA**.
- **ECMA International** standardisiert **JavaScript** unter dem Namen **ECMAScript**.

Wichtig: HTML lag frueher auch beim W3C. Heute ist die WHATWG die primaere Instanz fuer HTML, waehrend das W3C fuer andere zentrale Webstandards weiter wichtig bleibt.

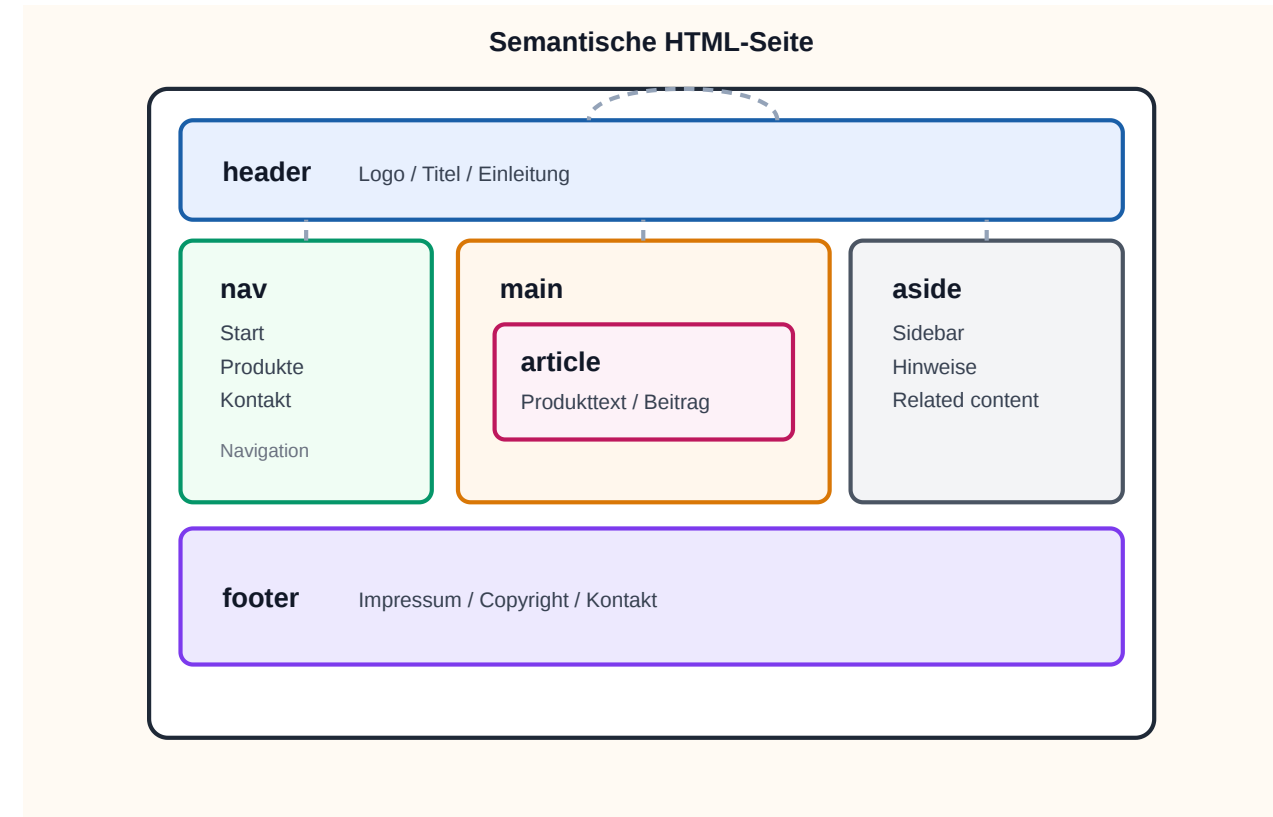
Browser implementieren diese Spezifikationen. Deshalb funktioniert dieselbe Seite grundsaeztlich in Chrome, Firefox und Safari.



Semantische Elemente in HTML5

HTML5 hat generische Container (<div>,) durch bedeutungstragende Elemente ergänzt:

Element	Bedeutung
<header>	Kopfbereich einer Seite oder Sektion
<nav>	Navigationsbereich
<main>	Hauptinhalt der Seite (einmalig)
<article>	Eigenständiger Inhalt (Blogpost, Produkt)
<section>	Thematischer Abschnitt
<aside>	Ergänzender Inhalt (Sidebar, Hinweis)
<footer>	Fußbereich
<figure> / <figcaption>	Bild mit Beschriftung



Faustregel Wenn ein Element eine **inhaltliche Bedeutung** hat, gibt es dafür ein semantisches HTML-Element. <div> ist für Layout-Container ohne eigene Bedeutung.

Semantik kann mehr: Microformats und Microdata

Semantik in HTML endet nicht bei `<header>` und `<article>`. Für maschinenlesbare Zusatzinformationen gibt es auch Microformats und Microdata.

```
<a class="h-card" href="/team/max">
  <span class="p-name">Max Mustermann</span>
  
</a>

<article itemscope itemtype="https://schema.org/Person">
  <h1 itemprop="name">Max Mustermann</h1>
  <p itemprop="jobTitle">Data Engineer</p>
</article>
```

- **Microformats** nutzen Klassen wie `h-card`, `p-name`, `u-photo`
- **Microdata** nutzt `itemscope`, `itemtype`, `itemprop`
- Beide ergänzen vorhandenes HTML statt es zu ersetzen

Details: [MDN Microformats](#) und [MDN Microdata](#)

Wofür braucht man das?

Microformats und Microdata sind nützlich, wenn HTML nicht nur für Menschen lesbar sein soll, sondern auch für Programme, die Inhalte erkennen, zusammenfassen oder weiterverarbeiten.

Typische Nutzung

- **Suchmaschinen:** Produkte, Personen, Organisationen, Events, Bewertungen
- **Aggregatoren und Crawler:** Inhalte automatisch aus Seiten extrahieren
- **Interne Systeme:** strukturierte Daten ohne separates JSON-Format
- **Semantic Web / schema.org:** zusätzliche Bedeutung direkt im HTML

Was die Nutzung bringt

- bessere maschinelle Erkennbarkeit von Inhalten
- mögliche Rich Results / strukturierte Vorschau in Suchmaschinen
- weniger Parsing-Hacks in externen Tools
- HTML bleibt die primäre Quelle statt nur ein Container für separate Metadaten

Microformats

- oft für Personen, Kontakte, Termine, Social Links
- leichtgewichtig und direkt lesbar
- Beispiel: h-card, h-event, p-name

Microdata

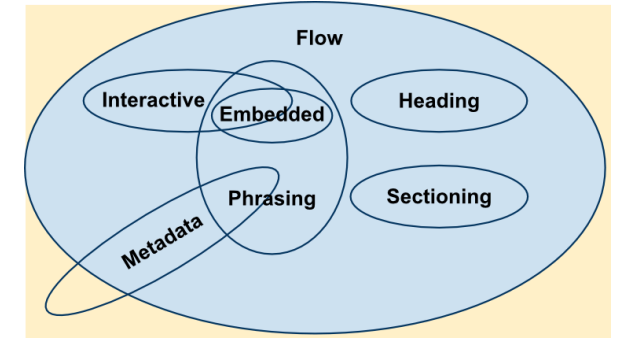
- häufig für strukturierte Entitäten wie Person, Product, Event
- enger an Vokabulare wie schema.org gebunden
- Beispiel: `itemscope`, `itemtype`, `itemprop`

Kurz gesagt: Microformats/Microdata ergänzen Semantik dort, wo reine Textstruktur nicht reicht.

HTML Content Categories

HTML-Elemente gehören zu formalen Kategorien. Diese Kategorien sagen nicht nur, **was** ein Element bedeutet, sondern auch **wo** es im Dokument eingesetzt werden soll.

Kategorie	Idee	Typische Elemente
Flow Content	allgemeine Bausteine im <body>	<div>, <p>, <section>,
Phrasing Content	Inhalt im laufenden Text	, <a>, ,
Heading Content	Überschriftenstruktur	<h1> bis <h6>
Sectioning Content	neue inhaltliche Abschnitte	<article>, <section>, <nav>, <aside>
Embedded Content	eingebettete externe Inhalte	, <video>, <iframe>
Interactive Content	direkt bedienbare Elemente	<a>, <button>, <input>
Metadata Content	Dokument-Metadaten im <head>	<title>, <meta>, <link>



Warum diese Kategorien wichtig sind

Die wichtigste Regel

Phrasing Content darf in laufendem Text stehen.

Flow Content ist allgemeiner und darf viele Blockelemente enthalten.

Deshalb ist ein `<span>` in einem `<p>` normal, ein `<div>` in einem `<p>` aber **invalides HTML**.

```
<p>Gültig: <span>Inline-Inhalt</span></p>  
<p>Ungültig: <div>Block-Inhalt</div></p>
```

Praktischer Nutzen

- hilft beim **korrekten Verschachteln** von Elementen
- erklärt Validator-Fehler verständlicher
- verhindert stille Browser-Korrekturen
- macht klar, welche Elemente für Text, Abschnitte oder Interaktion gedacht sind

Browser reparieren ungültiges HTML oft stillschweigend. Genau das macht solche Fehler im Layout später schwer nachvollziehbar.

Formulare: Die Schnittstelle zum Server

Unser Online-Shop braucht ein Registrierungsformular. HTML bietet dafür eingebaute Validierung:

```
<form action="/api/register" method="POST">
  <label for="email">E-Mail:</label>
  <input type="email" id="email"
        name="email" required>

  <label for="password">Passwort:</label>
  <input type="password" id="password"
        name="password" minlength="8" required>

  <button type="submit">Registrieren</button>
</form>
```

Attribut	Wirkung
required	Feld muss ausgefüllt sein
type="email"	Browser prüft E-Mail-Format
minlength	Mindestlänge des Passworts
pattern	Regex-basierte Validierung

Diskussionsfrage: Der Browser validiert clientseitig. Reicht das? Was passiert, wenn jemand das Formular nicht über den Browser, sondern per `curl` oder `Fetch API` abschickt?

HTML5 als Plattform

HTML5 hat den Browser von einem Dokumentbetrachter zu einer **Laufzeitumgebung** erweitert:

Fähigkeit	API / Element	Typischer Einsatz
Multimedia	<audio>, <video>	Streaming, Podcasts
Zeichenfläche	<canvas>, WebGL	Spiele, Datenvisualisierung
Lokaler Speicher	localStorage, sessionStorage, IndexedDB	Offline-Daten, Einstellungen
Hintergrundberechnung	Web Workers	CPU-intensive Aufgaben ohne UI-Blockade
Echtzeitkommunikation	WebSockets	Chat, Live-Updates
Geolocation	Geolocation API	Kartenanwendungen
Push-Benachrichtigungen	Notification API + Service Worker	Progressive Web Apps

Architekturentscheidung: Je mehr Fähigkeiten im Browser verfügbar sind, desto mehr Logik **kann** ins Frontend verlagert werden. **Aber sollte sie das?** Ein Online-Shop, der Preise im Client berechnet, riskiert Manipulation. Ein Offline-fähiger Warenkorb dagegen verbessert die User Experience. Die Grenze zwischen Client und Server ist eine bewusste Architekturentscheidung.

Responsive Images

Ein Bild sollte nicht auf jedem Gerät gleich ausgeliefert werden. Bei `img` gibt es dafür drei zentrale Attribute:

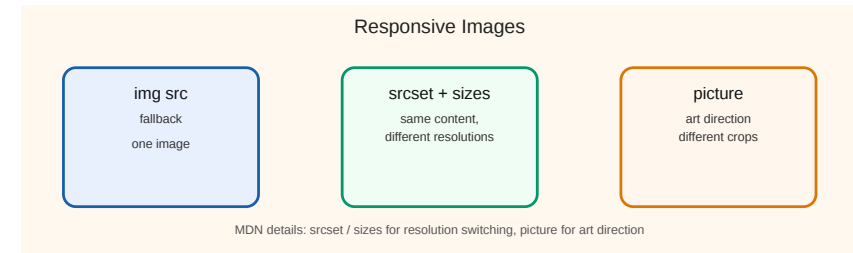
```

```

`src` ist das Fallback-Bild. Wenn der Browser `srcset` oder `sizes` nicht auswertet, lädt er mindestens diese Datei.

`srcset` listet Varianten derselben Bildinformation auf, meist in verschiedenen Breiten: `400w`, `800w`, `1600w`.

`sizes` beschreibt, wie breit das Bild im Layout ungefähr dargestellt wird. Erst damit kann der Browser aus `srcset`



sinnvoll auswählen.

- srcset + sizes = **Resolution Switching**
- Details: [MDN Responsive images](#)

Browser-Logik: sizes sagt, wie viel Platz das Bild im Layout bekommt; srcset sagt, welche Dateibreiten verfügbar sind; der Browser kombiniert beides mit der Pixeldichte des Geräts.

Responsive Images mit <picture>

Wenn sich nicht nur die Auflösung, sondern **der Bildausschnitt oder das Format** ändern soll, reicht `img` allein nicht mehr:

```
<picture>
  <source
    media="(max-width: 768px)"
    srcset="hero-mobile.webp">
  <source
    type="image/avif"
    srcset="hero-desktop.avif">
  
</picture>
```

`<source media="...">` wählt je nach Viewport oder anderen Media Queries unterschiedliche Bildvarianten.

Art Direction bedeutet: dieselbe Information wird für verschiedene Kontexte anders zugeschnitten oder komponiert, etwa engerer Crop auf Mobile.

Formatwahl bedeutet: der Browser kann moderne Formate wie AVIF oder WebP bevorzugen, wenn er sie unterstützt.

Das innere `<img>` bleibt Pflicht: Es ist Fallback, liefert das `alt`-Attribut und wird

- <picture> = **Art Direction** oder Format-Auswahl
- img[srcset][sizes] = **gleiche Bildinformation**, nur andere Auflösung
- Details: [MDN <picture>](#)

Kann HTML das allein? Progressive Enhancement

Bevor man CSS oder JavaScript einsetzt, lohnt die Frage: Was kann HTML nativ?

Anforderung	HTML allein?	Erfordert CSS?	Erfordert JS?
Navigation zwischen Seiten	<code><a href></code> — ja	—	—
Formulardaten an Server senden	<code><form action></code> — ja	—	—
E-Mail-Format validieren	<code><input type="email"></code> — ja	—	—
Bild mit Beschriftung	<code><figure></code> + <code><figcaption></code> — ja	—	—
Dreispaltiges Layout	nein	Grid/Flexbox	—
Hover-Effekt auf Button	nein	:hover Pseudo-Klasse	—
Daten vom Server nachladen ohne Seitenreload	nein	nein	<code>fetch()</code>
Drag & Drop im Warenkorb	nein	nein	Event Handling

Progressive Enhancement Beginne mit funktionierendem HTML. Füge CSS für Darstellung hinzu. Ergänze JavaScript nur wo nötig. Jede Schicht **erweitert**, statt die vorherige zu ersetzen.

Takeaway

HTML definiert die Struktur und Semantik von Web-Inhalten. Der Unterschied zwischen `<div>`-Suppe und semantischem HTML ist nicht kosmetisch — er bestimmt Barrierefreiheit, Auffindbarkeit und Wartbarkeit. Mit HTML5-APIs wird der Browser zur Plattform, was die Frage aufwirft, wie viel Logik ins Frontend gehört. HTML ist deshalb keine reine Markup-Sprache, sondern die Grundlage der Frontend-Architektur.

JavaScript — Die Sprache des Webs

Leitfrage: JavaScript ist die einzige Programmiersprache, die nativ in jedem Browser läuft. Was muss man über sie wissen, um Web-Systeme architektonisch zu verstehen?

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

```
document
├── html
│   ├── head
│   └── body
│       ├── h2 "Funkkopfhörer"
│       ├── p  "€ 79,99"
│       └── button "In den Warenkorb"
```

Produktseite unseres Online-Shops

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee

Der DOM ist die **Brücke zwischen HTML und JavaScript**. Jede Änderung am DOM wird sofort im Browser sichtbar.

JavaScript als Brücke zwischen HTML und CSS

JavaScript ist die **einzige** Frontend-Technologie, die sowohl den DOM (HTML) als auch das CSSOM (CSS) verändern kann. Das macht JS zum **Orchestrator**:

CSSOM bedeutet *CSS Object Model*: der interne Baum der CSS-Regeln, den der Browser aus Stylesheets und Inline-Styles aufbaut. Wenn JavaScript Klassen, Inline-Styles oder Custom Properties ändert, beeinflusst es damit nicht nur HTML, sondern auch die berechnete Darstellung.

```
// JS → HTML: DOM-Struktur ändern
const msg = document.createElement('div');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);

// JS → CSS: Klassen steuern Darstellung
button.classList.add('loading');      // CSS-Transition startet
button.classList.remove('loading');    // CSS-Transition endet

// JS → CSS: Custom Properties dynamisch setzen
document.documentElement.style.setProperty('--cart-count', '3');
```

Die Richtung der Abhängigkeit

```
HTML  ←— JavaScript —→ CSS
(DOM)  liest & ändert  (CSSOM)
```



- **HTML → JS:** HTML existiert ohne JS (Links, Formulare funktionieren). JS **erweitert** HTML.
- **CSS → JS:** CSS reagiert auf Zustände (:hover, :checked) ohne JS. JS **triggert** CSS-Übergänge über Klassen.
- **JS → beide:** Nur JS kann DOM und CSSOM gleichzeitig manipulieren — deshalb ist JS die Verhaltensschicht.

Konsequenz: Wann immer ein visueller Effekt auch per CSS lösbar ist (Transition, Animation, Hover), sollte CSS es übernehmen — JS steuert nur den **Auslöser** (Klasse setzen), nicht die **Animation** selbst.

Events: Auf Nutzeraktionen reagieren

JavaScript arbeitet **ereignisgesteuert**: Code wird ausgeführt, wenn etwas passiert. In unserem Online-Shop:

```
const button = document.querySelector('#add-to-cart');

button.addEventListener('click', async (event) => {
  // 1. Sofortiges visuelles Feedback
  button.textContent = 'Wird hinzugefügt...';
  button.disabled = true;

  // 2. Server kontaktieren (asynchron)
  const response = await fetch('/api/cart', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId: 42, quantity: 1 })
  });

  // 3. UI aktualisieren basierend auf Serverantwort
  if (response.status === 201) {
    button.textContent = '✓ Im Warenkorb';
    updateCartBadge();
  }
});
```

Diskussionsfrage: Der Button wird sofort deaktiviert und der Text geändert, **bevor** die Serverantwort eintrifft. Warum macht man das? Was passiert, wenn die Netzwerkverbindung abbricht?

Fetch API: Die Client-Seite von HTTP

Die Fetch API ist die Brücke zwischen Frontend (Vorlesung 03) und Protokoll (Vorlesung 04):

```
// GET: Produktliste laden
const response = await fetch('/api/products?category=electronics&sort=-price');
const products = await response.json();

// POST: Bestellung aufgeben
const result = await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ productId: 42, quantity: 1 })
});
if (result.status === 201) {
  const order = await result.json();
  window.location.href = `/orders/${order.id}`; // Weiterleitung
}
```

Fetch-Konzept	HTTP-Gegenstück
method: 'POST'	HTTP-Methode — definiert die Absicht
headers	HTTP Request Headers — Metadaten
response.status	HTTP-Statuscode — Ergebnis der Operation
response.json()	Body parsen — die Repräsentation der Ressource

Was kann schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');  
const product = await response.json();  
document.querySelector('.name').textContent = product.name;  
document.querySelector('.price').textContent = product.price + ' €';
```

Probleme und Lösungen

Problem	Was passiert	Lösung
Server nicht erreichbar	fetch wirft Exception	try/catch + Fehlermeldung anzeigen
404 Not Found	response.ok ist false, .json() liefert Fehler-Body	Status prüfen vor .json()
Langsame Verbindung	UI zeigt alte/keine Daten	Loading-Indicator anzeigen
Response ist kein JSON	.json() wirft Exception	Content-Type prüfen
DOM-Element existiert nicht	querySelector liefert null → Fehler	Null-Check oder optionale Verkettung (?.)

Robuste Fehlerbehandlung ist keine Kür — sie ist der Unterschied zwischen einem Prototyp und einem Produktivsystem.

Warum Frameworks existieren: Das Skalierungsproblem

Unser Warenkorb-Button funktioniert mit `querySelector` und `addEventListener`. Was passiert, wenn die Anwendung wächst?

```
// 5 Zeilen für einen Button – übersichtlich
button.addEventListener('click', () => { badge.textContent = '3'; });

// Aber: 50 Produkte, Filter, Sortierung, Pagination, Warenkorb-Sidebar...
// → Hunderte querySelector-Aufrufe, manuelles DOM-Tracking, Zustandschaos
```

Das Problem imperativer DOM-Manipulation

Komplexität	Vanilla JS	Framework (React/Vue)
Ein Button	Einfach, direkt	Overhead — zu viel Abstraktion
10 interaktive Elemente	Machbar, wird unübersichtlich	Komponenten helfen
Komplexe SPA (Shop, Dashboard)	Zustand und DOM divergieren	Deklarativ: UI = f(state)

Der konzeptionelle Unterschied

- **Imperativ** (Vanilla JS): „Finde Element X, ändere Property Y, füge Z ein“
- **Deklarativ** (React/Vue): „So soll die UI aussehen, wenn der Zustand S ist“ — Framework berechnet die DOM-Änderungen

Frameworks lösen nicht ein Syntax-Problem, sondern ein **Zustandsmanagement-Problem**: Wie bleibt die UI konsistent, wenn sich Daten ändern?

JavaScript-Ökosystem: Überblick

Bereich	Technologie	Rolle
Browser-Frameworks	React, Vue.js, Angular, Svelte	Deklarative, komponentenbasierte UI
Server-Runtime	Node.js, Deno, Bun	Dasselbe Event-Loop-Modell serverseitig
Full-Stack Frameworks	Next.js, Nuxt.js, SvelteKit	Server-Rendering + Client-Interaktion
TypeScript	Typisierte Obermenge von JS	Statische Typprüfung für große Codebases
Build-Tools	Vite, webpack, esbuild	Bündelung, Transpilation, Optimierung

Die Wahl des Frameworks ist eine **Architekturentscheidung** — sie bestimmt, wie Zustand verwaltet, Code organisiert und die Anwendung deployt wird.

Takeaway

JavaScript ist die **Verhaltensschicht** des Webs und zugleich der Orchestrator: Es ist die einzige Technologie, die sowohl DOM (HTML) als auch CSSOM (CSS) verändern kann. Das Ausführungsmodell — single-threaded mit Event Loop — ermöglicht responsive UIs trotz Netzwerk-Latenz. Die Fetch API verbindet Frontend mit Backend. Für Web Engineering sind drei Konzepte entscheidend: das **Event-Loop-Modell**, die **Abhängigkeitsrichtung** (JS erweitert HTML/CSS, nicht umgekehrt) und die **Grenze zu Frameworks** (ab welcher Komplexität lohnt deklarative UI).

CSS — Layout und Responsive Design

Leitfrage: Warum braucht das Web eine eigene Sprache für Darstellung — und wo endet die Verantwortung von CSS, wo beginnt die von JavaScript?

Welche Farbe hat der Text?

```
<p id="info" class="highlight">Willkommen im Shop</p>
```

```
p      { color: blue; }  
.highlight { color: green; }  
#info   { color: red; }  
p.highlight { color: orange; }
```

Aufgabe: Der Paragraph matcht alle vier Selektoren. Welche Farbe wird angezeigt — und warum?

Auflösung: Spezifität entscheidet

Selektor	Spezifität (ID, Klasse, Element)	Farbe
p	(0, 0, 1)	blue
.highlight	(0, 1, 0)	green
p.highlight	(0, 1, 1)	orange
#info	(1, 0, 0)	red

ID schlägt Klasse schlägt Element. In großen Projekten führen Spezifitätskonflikte zu unerwartetem Verhalten — deshalb setzen moderne Projekte auf Konventionen (BEM) oder komponentenbezogenes CSS (Scoped Styles).

Warum existiert CSS als eigene Technologie?

In den 1990ern wurde Darstellung direkt in HTML geschrieben: `<font color="red" size="5">`. Das führte zu einem fundamentalen Problem:

Problem	Ohne CSS	Mit CSS
Redesign	Jede HTML-Seite einzeln ändern	Ein Stylesheet ändern → alle Seiten betroffen
Verschiedene Geräte	Eine Version für alle	Verschiedene Stylesheets: Screen, Print, Mobile
Barrierefreiheit	Visuelles Layout = Dokumentstruktur	Screenreader ignoriert CSS, liest Struktur
Teamarbeit	Designer ändert HTML	Designer ändert CSS, Entwickler ändert HTML

Das Kernkonzept: Ein HTML, viele Darstellungen

Dieselbe Shop-Seite kann mit **verschiedenem CSS** als Desktop-Layout, als mobile Ansicht, als Druckversion oder im Dark Mode erscheinen — ohne das HTML zu ändern. Das ist kein Feature, sondern der **Grund, warum CSS als separate Sprache existiert**.

Normal Flow zuerst



- HTML-Elemente folgen zunächst dem **normal flow**
- **Block-Elemente** beginnen auf einer neuen Zeile und nehmen typischerweise die volle verfügbare Breite ein
- **Inline-Elemente** bleiben in derselben Zeile und nehmen nur so viel Platz ein wie ihr Inhalt braucht
- Erst danach greifen `display`, `position`, Flexbox oder Grid als bewusste Abweichungen vom Standard

So sieht das aus

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>  
<div>Dieses Block-Element steht in einer eigenen Zeile.</div>
```

Im Browser: Der Absatztext läuft in einer Zeile, das `span` bleibt im Textfluss, und das `div` startet darunter auf einer neuen Zeile.

CSS ist deklarativ, nicht imperativ

CSS sagt nicht „zeichne ein Rechteck bei Pixel 200,300“. CSS sagt „dieses Element soll 80% breit sein, zentriert, mit 1rem Abstand“. Der **Browser** berechnet die konkreten Pixel. Das ist fundamental anders als Canvas/JavaScript-Zeichnen und der Grund, warum CSS-Layouts automatisch auf verschiedene Bildschirmgrößen reagieren können.

CSS in Kurzform

CSS besteht aus **Selektoren** und **Deklarationen**:

```
p.highlight {  
  color: orange;  
  font-weight: 600;  
}
```

- **Selektor:** `p.highlight` sagt, *welche* Elemente gemeint sind
- **Deklaration:** `color: orange;` sagt, *wie* sie aussehen sollen
- **Regelblock:** alles zwischen `{ ... }`

Was CSS auflöst

- **Kaskade:** mehrere Regeln können denselben Stil setzen
- **Spezifität:** manche Selektoren gewinnen bei Konflikten
- **Vererbung:** manche Eigenschaften wandern von Eltern zu Kindern

Merksatz: CSS beschreibt Stil als Regeln, nicht als Pixelbefehle.

Pseudo-Klassen



Pseudo-Klassen sind Selektoren für **Zustände** oder **Positionen** eines Elements. Sie beginnen mit `:` und hängen an einen normalen Selektor an.

```
a:hover {  
  text-decoration: underline;  
}  
  
input:focus {  
  border-color: #1a5fa8;  
}
```

- `:hover` trifft ein Element, wenn der Mauszeiger darüber liegt
- `:focus` trifft ein Element, wenn es per Tastatur oder Klick aktiv ist
- Pseudo-Klassen beschreiben also nicht *was* ein Element ist, sondern *in welchem Zustand* es gerade ist

Zwei typische Beispiele:

- `button:hover` für Hover-Effekte
- `input:focus` für fokussierte Formularfelder

Was CSS ohne JavaScript kann

Bevor man JavaScript für visuelle Effekte einsetzt, lohnt die Frage: Kann CSS das allein?

Anforderung	CSS-Lösung	JS nötig?
Hover-Effekt auf Produktkarte	:hover Pseudo-Klasse	nein
Warenkorb-Button-Animation	transition + :active	nein
Dark Mode	@media (prefers-color-scheme: dark)	nein
Formularfeld-Validierung visuell	:valid, :invalid, :focus	nein
Akkordeon (auf/zuklappen)	<details> + CSS	nein
Smooth Scroll	scroll-behavior: smooth	nein
Drag & Drop sortieren	—	ja
Daten vom Server nachladen	—	ja

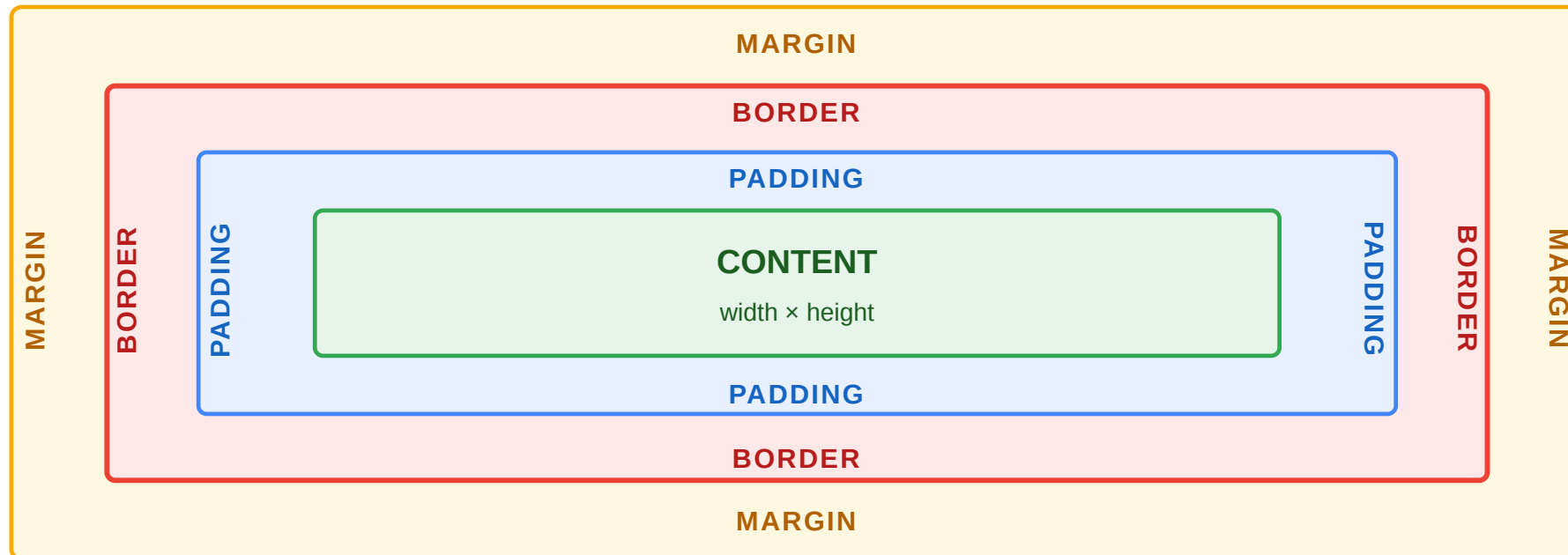
Die Grenze

CSS reagiert auf **Zustände** (:hover, :checked, :focus, @media) und **animiert Übergänge**. Sobald **neuer Zustand erzeugt** werden muss (Daten laden, DOM-Elemente erstellen, Server kontaktieren), braucht man JavaScript. JavaScript sollte den **Auslöser** steuern (Klasse setzen), CSS die **Darstellung** (Transition/Animation).

CSS-Layouts: Das Box-Modell

HTML Elemente müssen nun positioniert werden. Dazu wird jedes HTML-Element als eine Box betrachtet: **Content** → **Padding** → **Border** → **Margin**

Die häufigste Fehlerquelle: `width: 200px` bedeutet standardmäßig nur den Content — Padding und Border kommen **obendrauf**. Erst `border-box` macht `width` zum Gesamtmaß.



Layoutmodelle im Überblick

Nicht jedes CSS-Layoutproblem ist dasselbe. Die wichtigsten Modelle beantworten unterschiedliche Fragen:

Normal Flow

Default-Verhalten von HTML:

- Block-Elemente stapeln sich untereinander
- Inline-Elemente fließen im Text
- Ausgangspunkt für jedes Layout

Grid

Organisiert Elemente in **Zeilen und Spalten**:

- zweidimensional
- explizite Rasterstruktur
- ideal für Seitenlayout, Dashboards, Galerien

Flexbox

Verteilt Elemente entlang **einer Achse**:

- horizontal oder vertikal
- mit Abstand, Reihenfolge und Ausrichtung
- ideal für Reihen, Menüs, Toolbars

Faustregel

- **Flow**: Was passiert ohne Layoutanweisung?
- **Flexbox**: Wie verteilen wir Inhalt entlang einer Achse?
- **Grid**: Wie definieren wir ein Raster?

Normal Flow: Das Default-Layout des Browsers

Bevor wir Flexbox oder Grid einsetzen, sollten wir verstehen, was HTML **ohne besondere Layoutregeln** tut:

Block-Elemente beginnen typischerweise in einer neuen Zeile und nehmen die verfügbare Breite ein.

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>  
<div>Dieses Block-Element steht darunter.</div>
```

Inline-Elemente bleiben im laufenden Text und belegen nur so viel Platz wie ihr Inhalt braucht.

Absatz **Inline-Text** nach dem Wort.

Dieses Block-Element steht darunter.

Flow ist deshalb wichtig, weil Flexbox und Grid immer bewusst vom Normalfall abweichen.

Flexbox: Verteilung entlang einer Achse

Flexbox ist das richtige Werkzeug, wenn Elemente in **einer Zeile oder Spalte** organisiert werden sollen:

```
.nav {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 1rem;  
}
```

Logo

Produkte

Warenkorb

Typische Fragen:

- nebeneinander oder untereinander?
- wie viel Abstand?
- links, mittig oder rechts ausrichten?

Flexbox ist ideal für:

- Navigation
- Button-Gruppen
- Kartenreihe
- Toolbar oder Header-Leiste

Grid: Ein explizites 2D-Raster

Grid ist sinnvoll, wenn das **Layout selbst** die Struktur vorgibt.

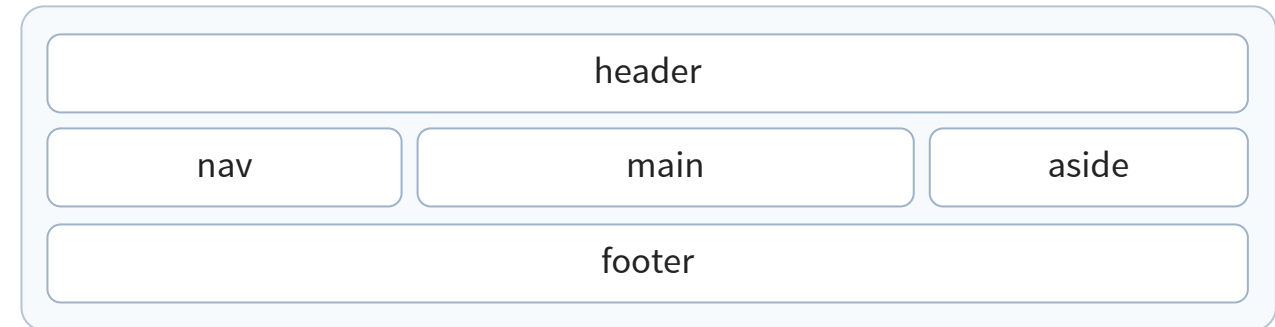
Explizit bedeutet hier: In CSS werden **Spalten**, **Zeilen** und oft auch die **Platzierung der Bereiche** vorgegeben.

```
.shop-layout {  
  display: grid;  
  grid-template-columns: 240px 1fr 220px;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header header"  
    "nav   main   aside"  
    "footer footer footer";  
  gap: 1rem;  
}
```

1fr bedeutet: nimm **einen Anteil des verbleibenden freien Platzes**.

"header header header" bedeutet: Der Bereich header belegt in dieser Zeile **alle drei Spalten**.

grid-template-columns definiert die **Spalten**.
grid-template-rows definiert die **Zeilen**.
grid-template-areas ordnet benannte Bereiche im Raster an.



Bei Grid entsteht zuerst das Raster. Danach werden die Elemente in dieses Raster eingeordnet.

Responsives Grid ohne starre Breakpoints

Ein Grid kann responsiv werden, ohne für jedes Gerät ein eigenes Layout zu definieren:

```
.product-grid {  
  display: grid;  
  gap: 1rem;  
  grid-template-columns:  
    repeat(auto-fit, minmax(220px, 1fr));  
}
```

`minmax(220px, 1fr)` bedeutet:

- nie schmaler als 220px
- sonst flexibel wachsen

`auto-fit` versucht so viele Spalten wie möglich in die Zeile zu legen.

Produkt A

Produkt B

Produkt C

⋮

Damit entsteht ein fluides Raster:

- auf schmalen Screens wenige Spalten
- auf breiten Screens mehr Spalten
- ohne duplizierte Layouts

Beispiel: Produktdetailseite mit Bild und Text

Situation: Links steht das Produktbild, rechts Beschreibung und Kaufbutton. Auf dem Smartphone soll das Bild oben und der Text darunter erscheinen.

Mögliche Wahl: Flexbox

- funktioniert für einfache Zeile/Spalte
- gut bei linearen Anordnungen
- weniger explizit bei komplexer Umordnung

Mögliche Wahl: Grid

- beschreibt Desktop als "img text"
- beschreibt Mobile als "img" / "text"
- Layout bleibt als Struktur sichtbar

Welche Wahl ist hier besser?

Flexbox passt, wenn

- nur eine Hauptachse wichtig ist
- die Elemente einfach verteilt werden
- Reihenfolge und Struktur weitgehend linear bleiben

Grid passt, wenn

- Zeilen und Spalten gemeinsam wichtig sind
- Bereiche explizit benannt werden sollen
- responsives Umschalten strukturell klar bleiben soll

Faustregel: Flexbox, wenn der Inhalt verteilt wird. Grid, wenn das Layout als Raster vorgegeben wird.

Takeaway

CSS existiert als eigene Sprache, weil dasselbe HTML in **verschiedenen Kontexten** verschieden dargestellt werden muss — Screen, Print, Mobile, Dark Mode. CSS ist **deklarativ**: Man beschreibt Constraints, der Browser berechnet Pixel. Spezifität bestimmt, welche Regel gewinnt. Flexbox und Grid lösen unterschiedliche Verteilungsprobleme (Inhalt verteilen vs. Raster vorgeben). Responsive Design mit Mobile-First ist Voraussetzung. Die wichtigste Grenzfrage: **Kann CSS das allein** (Hover, Transition, Media Query), oder braucht es JavaScript? Nur wenn neuer Zustand erzeugt werden muss, ist JS die richtige Schicht.

Web APIs — die Schnittstellen des Browsers

Leitfrage: Was sind Web APIs — und warum gehören sie nicht zur JavaScript-Sprache selbst, sondern zum Browser?

Web APIs in einem Satz

Web APIs sind Browser-Schnittstellen, die JavaScript zusätzliche Fähigkeiten geben: den DOM verändern, Netzwerke ansprechen, Daten speichern, Gerätefunktionen nutzen oder Medien steuern.

Typische Browser-APIs

- DOM API
- Fetch API
- Web Storage API
- Geolocation API
- Canvas / WebGL
- Web Workers

Warum das wichtig ist

- JavaScript allein beschreibt die Sprache
- Web APIs machen sie im Browser praktisch nutzbar
- viele Web-Funktionen sind erst durch diese APIs möglich

Beispiel: `fetch()` lädt Daten, `localStorage` speichert Zustand, DOM APIs aktualisieren die Oberfläche.

Wofür nutzt man sie?

Aufgabe	Beispiel-API
HTML oder CSS zur Laufzeit ändern	DOM API
Daten vom Server laden	Fetch API
Zustand im Browser speichern	Web Storage API
Standorte abfragen	Geolocation API
Lange Berechnungen auslagern	Web Workers

Kernaussage: Web APIs sind die Schnittstelle zwischen JavaScript und den Fähigkeiten des Browsers.

Grenze: Nicht jede API ist Teil von JavaScript selbst. Viele Funktionen kommen vom Browser, nicht von der Sprache.

Fetch API im Detail: Request, Response, AbortController

```
const controller = new AbortController();

// Request mit Timeout nach 5 Sekunden abbrechen
const timeout = setTimeout(() => controller.abort(), 5000);

try {
  const response = await fetch('/api/products', {
    method: 'GET',
    headers: { 'Accept': 'application/json' },
    signal: controller.signal,
  });

  if (!response.ok) throw new Error(`HTTP ${response.status}`);

  const products = await response.json();
} catch (err) {
  if (err.name === 'AbortError') console.log('Request abgebrochen');
  else console.error('Netzwerkfehler', err);
} finally {
  clearTimeout(timeout);
}
```

Konzept	Zweck
Request / Response	Objektmodell für HTTP-Nachrichten
headers	Metadaten: Content-Type, Accept, Authorization
AbortController	Laufende Requests abbrechen (Timeout, Navigation)
response.ok	true bei Status 200–299

Web Storage: localStorage, sessionStorage, Cookies

API	Lebensdauer	Größe	Zugriff
localStorage	Persistent (bis manuell gelöscht)	~5–10 MB	Nur Client (JS)
sessionStorage	Nur aktuelle Browser-Session	~5–10 MB	Nur Client (JS)
Cookies	Konfigurierbar (Session oder Ablaufdatum)	~4 KB	Client + automatisch an Server
IndexedDB	Persistent	Gigabyte-Bereich	Client (asynchron)

```
// localStorage: einfache Key-Value-Speicherung
localStorage.setItem('cart', JSON.stringify(cartItems));
const cart = JSON.parse(localStorage.getItem('cart') || '[]');

// sessionStorage: nur für die aktuelle Tab-Session
sessionStorage.setItem('wizard-step', '3');

// Cookies: werden bei jedem HTTP-Request mitgesendet
document.cookie = 'theme=dark; max-age=31536000; SameSite=Strict';
```

Wann was?

- **Cookies** für alles, was der **Server wissen muss** (Session-ID, Authentifizierung)
- **localStorage** für Client-State, der **über Sessions** erhalten bleiben soll (Warenkorb, UI-Einstellungen)
- **sessionStorage** für temporären State, der **nur im aktuellen Tab** gilt (Wizard-Fortschritt)
- **IndexedDB** für große Datenmengen oder Offline-Anwendungen



History API: Navigation ohne Seiten-Reload

Single-Page Applications navigieren ohne vollständigen Seiten-Reload. Der Browser-Zurück-Button muss trotzdem funktionieren. Die **History API** macht das möglich:

```
// URL ändern, ohne neu zu laden
history.pushState({ page: 'products' }, '', '/products');

// Neue Inhalte fetch'en und rendern
const data = await fetch('/api/products').then(r => r.json());
renderProducts(data);

// Zurück-Button abfangen
window.addEventListener('popstate', (event) => {
  if (event.state?.page === 'products') renderProducts(/* ... */);
});
```

Methode / Event	Zweck
history.pushState(state, '', url)	URL ändern, Eintrag zum Verlauf hinzufügen
history.replaceState(...)	URL ändern, ohne neuen Eintrag
popstate Event	Browser-Zurück-/Vorwärts-Button

Web Workers vs Service Workers

Beide laufen in einem **separaten Thread** außerhalb des Haupt-Event-Loops — aber mit unterschiedlichen Zwecken.

Aspekt	Web Worker	Service Worker
Zweck	CPU-intensive Berechnungen auslagern	Netzwerk-Proxy, Offline-Support, Push
Lebensdauer	Gebunden an Tab/Seite	Persistent, auch wenn Tab geschlossen
DOM-Zugriff	Nein	Nein
Fetch-Interception	Nein	Ja — fängt alle Netzwerk-Requests ab
Typischer Einsatz	Bildverarbeitung, Datenanalyse, Kryptographie	Progressive Web Apps, Caching, Push-Benachrichtigungen

```
// Web Worker: teure Berechnung auslagern
const worker = new Worker('compute.js');
worker.postMessage({ data: largeDataset });
worker.onmessage = (e) => displayResult(e.data);

// Service Worker: Fetch abfangen für Offline-Support
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```



Takeaway

Web APIs sind die **Fähigkeiten**, die der Browser JavaScript zur Verfügung stellt — nicht Teil der Sprache selbst, sondern Erweiterungen. Die wichtigsten Kategorien sind DOM-Manipulation, Netzwerk (Fetch), Speicher (localStorage/Cookies), Hintergrund-Threads (Web/Service Workers) und History. Welche API wofür eingesetzt wird, ist eine Architekturfrage mit Konsequenzen für Sicherheit, Performance und Offline-Fähigkeit.

Das Zusammenspiel im Browser

Leitfrage: Wie arbeiten HTML, CSS und JavaScript im Browser zusammen — und warum ist das Verständnis dieser Pipeline für Performance und Architektur entscheidend?

Was passiert, wenn Sie „In den Warenkorb“ klicken?

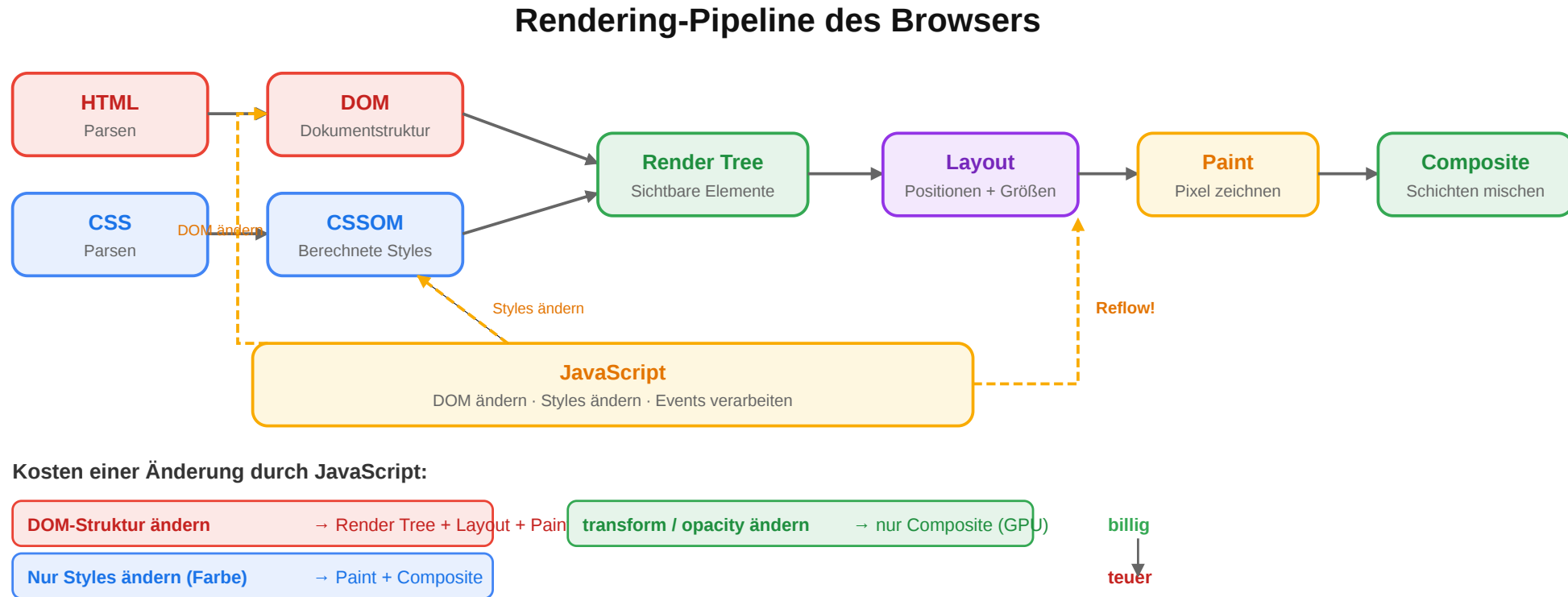
Aufgabe: Skizzieren Sie den Ablauf: Welche Technologie (HTML, CSS, JavaScript) ist in welchem Schritt beteiligt? Was passiert sofort, was asynchron? Was sieht der Nutzer wann?

Ablauf (Skizze)

1. Nutzer klickt den Button → JavaScript reagiert auf das Event.
2. JavaScript aktualisiert DOM-Zustände (Badge, Button-Text).
3. CSS/Layout berechnet neues Rendering, der Nutzer sieht sofort Feedback.
4. JavaScript sendet asynchron POST `/api/cart` an den Server.
5. Server antwortet, JavaScript aktualisiert DOM erneut.
6. Browser führt Layout und Paint erneut aus.

Schlüsselbeobachtung: Das UI reagiert **sofort** (optimistisches Update), der Server wird **asynchron** kontaktiert. Das ist nur möglich, weil JavaScript non-blocking arbeitet.

Die Rendering-Pipeline des Browsers



Die Pipeline arbeitet auf zwei internen Strukturen: dem **DOM** für HTML und dem **CSSOM** für CSS. Erst daraus erzeugt der Browser den Render Tree und rechnet Layout, Paint und Compositing aus.

Performance: Die häufigsten Fehler

Problem	Ursache	Lösung
Render-Blocking CSS	Browser wartet auf gesamtes CSS, bevor er etwas anzeigt	Critical CSS inline, Rest asynchron laden
Parser-Blocking JS	<script> stoppt HTML-Parsing	defer oder async Attribut
Layout Thrashing	JavaScript liest und schreibt abwechselnd DOM-Properties	Reads bündeln, dann Writes bündeln
Große JS-Bundles	Zu viel JavaScript auf einmal geladen	Code Splitting, Lazy Loading
Unoptimierte Bilder	5 MB Produktbilder ohne Kompression	WebP/AVIF, responsive srcset, Lazy Loading

Kernprinzip

Performance-Probleme im Web entstehen fast immer an der **Schnittstelle** von HTML, CSS und JavaScript — nicht innerhalb einer einzelnen Technologie. Deshalb muss man die Pipeline verstehen.

Core Web Vitals: Wie Google Performance misst

Google misst Performance anhand dreier Nutzer-zentrierter Metriken, die auch ins Suchranking einfließen:

Metrik	Was es misst	Ziel
LCP — Largest Contentful Paint	Zeit, bis das größte sichtbare Element gerendert ist	< 2,5 s
INP — Interaction to Next Paint	Verzögerung zwischen Klick/Tap und sichtbarer Reaktion	< 200 ms
CLS — Cumulative Layout Shift	Wie stark sich das Layout nach Laden verschiebt	< 0,1

Typische Verursacher

- **Schlechter LCP:** große, unoptimierte Hero-Bilder; render-blocking CSS; langsamer Server
- **Schlechter INP:** lange Haupt-Thread-Blockaden durch JavaScript
- **Schlechter CLS:** Bilder ohne width/height; nachgeladene Werbung oder Fonts verschieben Inhalt

Warum das architektonisch wichtig ist

Core Web Vitals sind keine Laborwerte — sie werden am **echten Nutzer** gemessen (Real User Monitoring über Chrome UX Report). Eine Seite kann im Labor schnell wirken und in der Praxis trotzdem schlechte Werte haben, wenn sie auf langsamen Geräten oder Netzwerken läuft.

Wie hat sich das Zusammenspiel verändert?

Ära	Muster	Zusammenspiel
Klassisch (2000er)	Server rendert HTML, CSS als separate Datei, JS für kleine Effekte	Strikte Trennung: HTML-Datei + CSS-Datei + JS-Datei
jQuery-Ära (2006–2015)	HTML vom Server, JS manipuliert DOM direkt	DOM als zentrale Schnittstelle
Komponentenbasiert (ab 2013)	React/Vue/Angular: HTML + CSS + JS pro Komponente	Trennung nach Verantwortung , nicht nach Technologie
Server-First (ab 2020)	Next.js/SvelteKit: Server rendert, Client hydriert	Erst HTML vom Server, dann JavaScript macht es interaktiv

Was bedeutet „Client hydriert“?

Der Server liefert bereits fertiges HTML an den Browser. Danach lädt JavaScript nach und **bindet Verhalten an dieses vorhandene HTML**:

- Event-Listener werden registriert
- Komponenten bekommen ihren interaktiven Zustand
- die Seite wird nutzbar, ohne dass der Browser alles neu rendern muss

Kurz: Hydration bedeutet, dass der Client das serverseitig gerenderte HTML „zum Leben erweckt“.

Diskussionsfrage

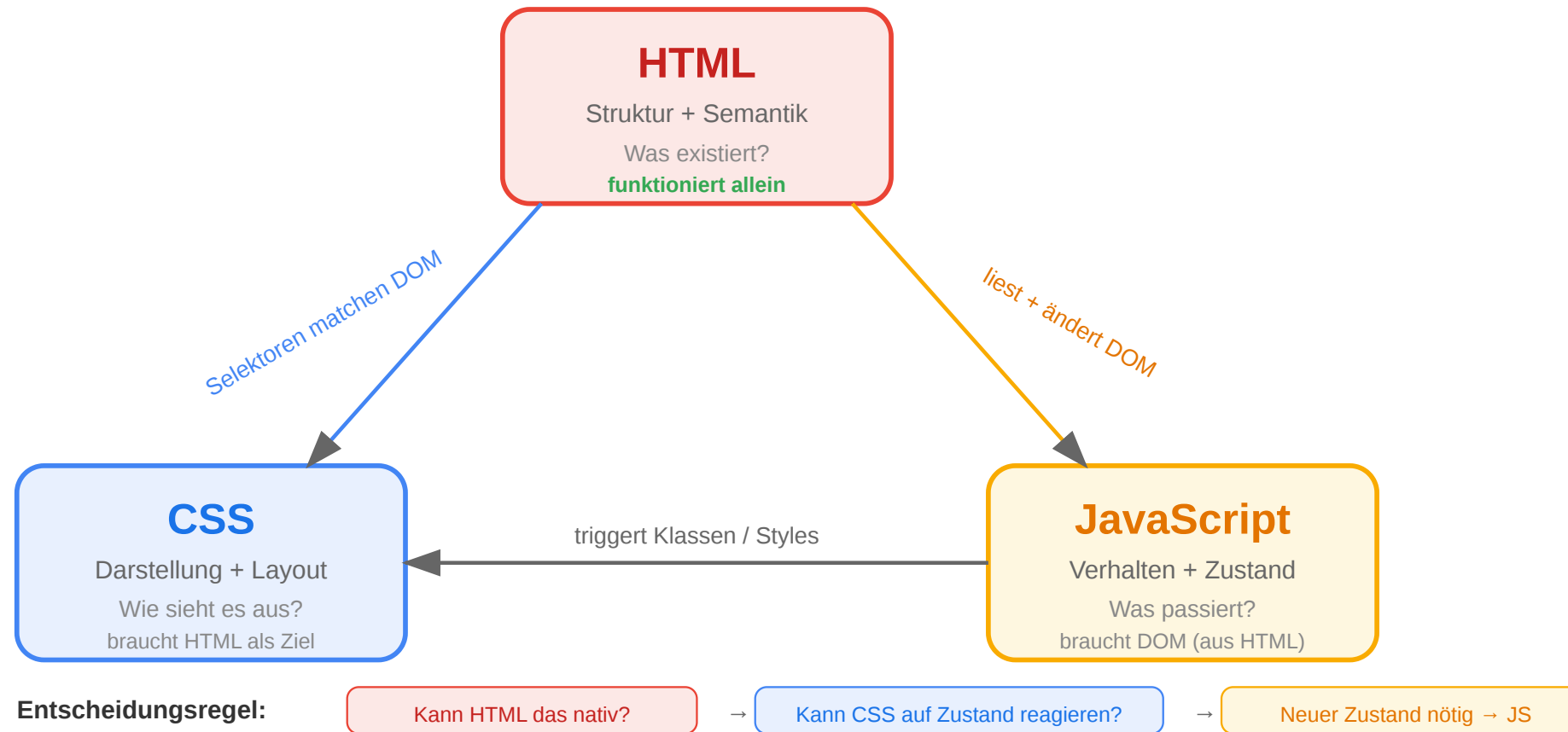
Die klassische Web-Lehre sagt: „Trenne HTML, CSS und JavaScript in separate Dateien." React bündelt alle drei **pro Komponente**. Ist das ein Widerspruch zum Prinzip der Separation of Concerns — oder eine bessere Umsetzung davon?

Einordnung

Beides stimmt: Die Trennung bleibt — aber auf einer anderen Ebene. Statt nach Technologie (HTML/CSS/JS) wird nach **fachlicher Verantwortung** getrennt (Produktkarte, Warenkorb, Navigation). Jede Komponente kapselt ihr Markup, ihren Stil und ihr Verhalten.

Das Technologie-Dreieck: Zusammenfassung

Das Technologie-Dreieck des Webs



HTML, CSS und JavaScript sind ein eng verzahntes System. Der Browser verarbeitet sie in einer festen Pipeline, die JavaScript jederzeit beeinflussen kann — mit Konsequenzen für Performance und Architektur. Moderne Frameworks organisieren das Zusammenspiel in Komponenten statt in Dateien. Das grundlegende Modell von DOM, Events und Rendering bleibt dasselbe.

Wrap-Up

- **HTML** definiert Struktur und Semantik — die Grundlage für Barrierefreiheit, SEO und maschinelle Verarbeitung.
- **JavaScript** macht den Browser programmierbar — über DOM, Events und asynchrone Kommunikation mit dem Server.
- **CSS** trennt Darstellung von Struktur — Flexbox und Grid lösen das Layout-Problem, Responsive Design das Geräteproblem.
- Die drei Technologien sind **eng verzahnt**: Der Browser verarbeitet sie in einer Pipeline (Parse → Render Tree → Layout → Paint), die JavaScript jederzeit beeinflussen kann.

Die nächste Vorlesung vertieft das **Protokoll** (HTTP) und das **API-Design** (REST), das die Brücke zwischen Frontend und Backend bildet.